

Python in Excel: Your First Hour

Everything from today's workshop, on one page. Pin it next to your monitor.

1 Insert a PY() cell

- ▶ Ribbon: **Formulas** → **Insert Python**
- ▶ Type =PY(in any cell
- ▶ Shortcut: **Ctrl** **Alt** **Shift** **P**
- ▶ Run the cell with **Ctrl** **Enter** — not Enter alone
- ▶ Toggle output mode: **Ctrl** **Alt** **Shift** **M**

2 Load a table → DataFrame

- ▶ Convert your range to an Excel Table first (**Ctrl** **T**) and name it
- ▶ `xl()` is the bridge from worksheet to Python

```
sales_df = xl("sales[#All]", headers=True)
sales_df      # preview as a card
```

3 Profile a dataset in 30 seconds

<code>.head()</code>	first 5 rows	<code>.describe()</code>	summary stats	<code>.columns</code>	column names
<code>.tail()</code>	last 5 rows	<code>.describe(exclude='number')</code>		<code>len(df)</code>	row count
<code>.sample(5)</code>	5 random rows	<code>.isna().sum()</code>	missing per column	<code>df['col']</code>	single column (Series)
<code>.shape</code>	(rows, columns)	<code>.dtypes</code>	column types	<code>df[['a', 'b']]</code>	several columns

4 Count categories & group

```
sales_df['country'].value_counts()
sales_df['category'].value_counts(normalize=True)

# PivotTable, in code
sales_df.groupby('country')['unit_price'].mean()
sales_df.groupby(['country', 'category'])['unit_price'].mean()
sales_df.groupby('country')['unit_price'] \
    .agg(['mean', 'min', 'max'])
```

5 Filter, sort, chain

```
# Filter rows
us = df[df['country'] == 'US']

# Sort, highest first
df.sort_values('unit_price',
               ascending=False)

# Filter → group → sort
(df[df['country'] == 'US']
 .groupby('category')['unit_price']
 .mean()
 .sort_values(ascending=False))
```

6 One chart that earns gasps

```
sales_df['revenue'] = sales_df['unit_price'] * sales_df['quantity']

import seaborn as sns
sns.heatmap(sales_df.corr(numeric_only=True),
            annot=True, cmap='RdGy_r', center=0)

# Bonus — every numeric variable vs. every other
sns.pairplot(sales_df, hue='category')
```

⚠ THE CALC-ORDER RULE

Python in Excel runs cells **left-to-right, top-to-bottom** — like reading a page. If you define a variable in B2, you can reference it in any cell to the right in row 2, or in any row below. Referencing it from A1 returns #PYTHON. **Recovery rule:** put imports and variables in the top-left, build down and right.

THREE TO REMEMBER

1. Cells run top-left → bottom-right. 2. Cloud-only: no internet, no PY(). 3. Default output is Python Object — toggle to Excel Value to spill.

WHERE NEXT

Docs: learn.microsoft.com (search "Python in Excel"). **Go deeper:** *Python in Excel for Data Analytics* (Packt, 2026).

DEFAULT ENVIRONMENT

pandas, numpy, scipy, matplotlib, seaborn, statsmodels, scikit-learn. No internet packages or local file I/O.